

Guide du Projet

Système RAG (Retrieval-Augmented Generation)

Module : IA Générative

1 Contexte

Dans le cadre du module d'IA Générative, les étudiants ont étudié le concept de **RAG (Retrieval-Augmented Generation)**, qui permet de combiner la recherche d'information avec la génération de texte par un modèle de langage (LLM).

Contrairement à un simple chatbot, un système RAG s'appuie sur des données externes pour produire des réponses fiables et contextualisées.

Ce projet vise à mettre en pratique les notions suivantes :

- Documents et corpus
- Chunking
- Embeddings
- Indexation vectorielle
- Retriever
- Contexte et génération

2 Objectif du projet

L'objectif est de concevoir un système RAG capable de :

- Charger un ensemble de documents
- Transformer les données en embeddings
- Stocker les embeddings dans une base vectorielle
- Rechercher les informations pertinentes
- Générer des réponses à partir du contexte récupéré

Objectifs pédagogiques :

- Comprendre le pipeline RAG complet
- Manipuler des embeddings et des bases vectorielles
- Utiliser un LLM via API (ex : Groq)
- Concevoir un chatbot basé sur des données

3 Description du projet

Le projet consiste à développer une application capable de répondre à des questions en se basant sur un corpus de documents.

3.1 Fonctionnement

- L'utilisateur pose une question
- Le système recherche les documents pertinents
- Le contexte est construit
- Le LLM génère une réponse

3.2 Exigence clé

- Le système doit utiliser des données réelles
- Les réponses doivent être basées sur le corpus

4 Choix de projets (options)

4.1 Option 1 : RAG Simple (Obligatoire)

- Chargement de documents
- Chunking
- Embeddings
- Stockage dans FAISS ou ChromaDB
- Retrieval
- Génération avec LLM

4.2 Option 2 : RAG Amélioré

- Amélioration du prompt
- Re-ranking
- Mémoire conversationnelle
- Interface utilisateur simple

4.3 Option 3 : RAG Modulaire

- Séparation des composants :
 - ingestion
 - retrieval
 - génération
- Code structuré

4.4 Option 4 : Agentic RAG (optionnel)

- Intégration d'un agent
- Choix dynamique entre recherche et réponse

5 Contraintes techniques

- Utilisation d'un LLM (ex : Groq)
- Implémentation complète du pipeline RAG
- Utilisation d'une base vectorielle

5.1 Exigences spécifiques

- Gestion du chunking (taille et overlap)
- Qualité des embeddings
- Pertinence du retrieval
- Construction du prompt avec contexte

5.2 Technologies recommandées

- Python
- LangChain
- FAISS / ChromaDB
- APIs LLM (Groq)

5.3 Technologies optionnelles

- Streamlit / Gradio (interface)
- LangGraph

6 Livrables

6.1 1. Code source

- Pipeline fonctionnel
- Code structuré

6.2 2. Rapport

- Description du projet
- Architecture
- Explication du pipeline RAG
- Choix techniques
- Résultats et limites

6.3 Structure recommandée du rapport

- Introduction
- Objectif
- Description du système
- Pipeline RAG
- Implémentation
- Résultats
- Conclusion

6.4 3. Démonstration

- Présentation orale
- Test du système

7 Organisation du projet

- Travail en groupe de **maximum 3 étudiants**

8 Suivi du projet

Le projet sera suivi et validé progressivement au cours des séances. Chaque étape donne lieu à une validation par l'enseignant.

8.1 Séance 4 : Lancement du projet

- Formation des groupes (max 3 étudiants)
- Choix du corpus (PDF, texte, web...)
- Définition :
 - de l'objectif du projet
 - du cas d'usage (chatbot, assistant, recherche...)
 - des technologies utilisées (FAISS, ChromaDB, LangChain...)

Livable attendu :

- Description du projet (1 page)
- Description du corpus
- Schéma simple du pipeline RAG

8.2 Séance 5 : Première validation

- Chargement des documents
- Implémentation du chunking
- Génération des embeddings

Livable attendu :

- Code fonctionnel pour :
 - lecture des documents
 - découpage en chunks
 - création des embeddings
- Tests simples

8.3 Séance 3 : Deuxième validation

- Indexation dans une base vectorielle (FAISS ou ChromaDB)
- Implémentation du retriever
- Test de recherche de documents pertinents

Livable attendu :

- Système capable de :
 - stocker les embeddings
 - récupérer les documents pertinents

Recommandations :

- Tester différents paramètres (top-k, distance...)
- Vérifier la pertinence des résultats

8.4 Séance 4 : Troisième validation

- Intégration du LLM (ex : Groq)
- Construction du prompt avec contexte
- Génération de réponses basées sur les documents

Livrable attendu :

- Pipeline RAG complet fonctionnel :
 - question utilisateur
 - retrieval
 - génération

8.5 Séance 5 : Validation finale

- Amélioration du système
- Optimisation du prompt
- (optionnel) ajout d'une interface

Livrables attendus :

- Code final
- Rapport
- Démonstration du système

Présentation orale :

- Description du projet
- Explication du pipeline RAG
- Démonstration

8.6 Recommandations générales

- Tester plusieurs tailles de chunk et overlap
- Comparer différentes bases vectorielles (FAISS, ChromaDB)
- Améliorer le prompt pour guider le LLM
- Utiliser des données réelles et pertinentes
- Commencer simple puis améliorer progressivement

9 Conclusion

Ce projet permet de comprendre et implémenter un système RAG complet, combinant recherche d'information et génération de texte.